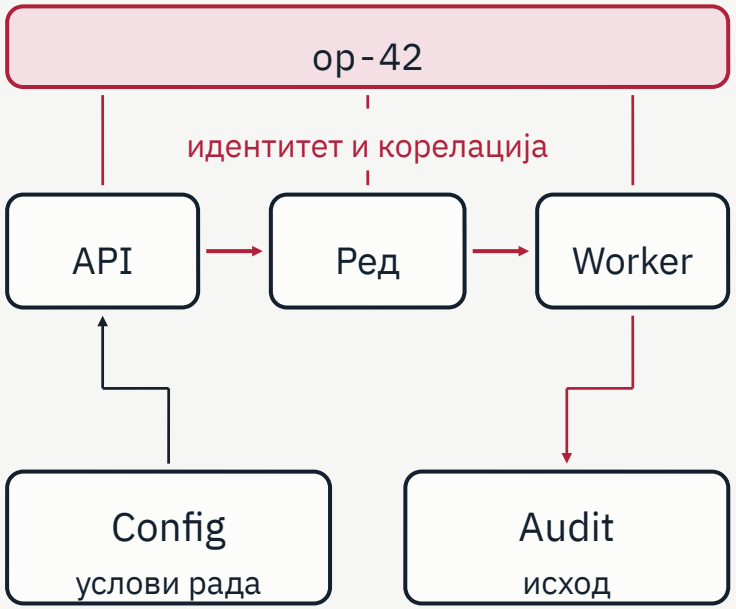
ПРИМЕЊЕНО СОФТВЕРСКО ИНЖЕЊЕРСТВО

ОСНОВЕ ДИСТРИБУИРАНОГ ПРОГРАМИРАЊА

Идентитет, audit, observability и конфигурација

Попречни слој за разумевање тока



Факултет техничких наука · Универзитет у Новом Саду

Нови Сад

ПРОВЕРА

Садржај презентације

01

Појмови и границе

Кључни појмови, одговорности компоненти и конкретан failure сценарио.

03–10

02

Дистрибуирани ток

Шема тока и гаранције које повезују компоненте система.

11–13

03

Код и failure тестови

Покретање примера, доменско правило, тест и видљиви исходи.

14–25

04

Примена и провера

Принципи, замке, задатак уз пример и примена на пројектну целину.

26–33

ПРВИ ДЕО

01

Појмови и границе

Кључни појмови, одговорности компоненти и конкретан failure сценарио.

01 · ПОЈМОВИ

Тема и проблем

Разумети како се операција прати кроз више компоненти без губитка контекста.

Када се ток прекине, засебни логови не дају одговор ко је покренуо радњу, шта се догодило и под којим условима.

НАПОМЕНА

Заједнички контекст и audit омогућавају реконструкцију тока, док конфигурација чува видљиве услове под којима систем ради.

01 · ПОЈМОВИ

Четири појма теме

Дистрибуирани систем мора моћи да одговори на четири питања: ко је покренуо радњу, шта се догодило, кроз које компоненте је ток прошао и под којим условима је радио.

01 **Идентитет**

ко покреће радњу

02 **Корелација**

један ток кроз компоненте

03 **Audit**

пословно значајна одлука

04 **Конфигурација**

услови рада система

01 · ПОЈМОВИ

Идентитет

Идентитет показује ко или шта је покренуло радњу. Без њега није могуће повезати захтев са одговорношћу и правом приступа.

01 Повезује захтев са одговорношћу и правом приступа.

02 Проверава се и улога и припадност мисији.

03 Нема одлуку о приступу без познатог идентитета.

01 · ПОЈМОВИ

Корелација

Исти корелациони идентификатор прати операцију кроз API, ред и worker. Он омогућава да се више техничких трагова прочита као један ток.

01 Исти идентификатор кроз API, ред и worker.

02 Спаја техничке трагове у један разумљив ток.

03 Не сме се регенерисати у сваком слоју.

01 · ПОЈМОВИ

Audit

Audit бележи одлуке које имају пословни или безбедносни значај: ко је шта променио, када и са којим исходом. Није замена за сваки технички лог.

01 Бележи пословно и безбедносно значајне одлуке.

02 Append-only и претражив по correlation id-у.

03 Није замена за сваки технички лог.

01 · ПОЈМОВИ

Конфигурација

Конфигурација одређује под којим условима компонента ради. Валидне вредности, извор и промена конфигурације морају бити видљиви тиму.

01 Одређује услове рада компоненте.

02 Валидан опсег и извор морају бити видљиви.

03 Промена конфигурације је следљива, не тиха.

Пример: захтев пролази кроз три компоненте

Корисник покреће операцију кроз API, API објављује поруку, а worker завршава обраду.

01 Корелациони идентификатор настаје на граници тока и преноси се даље.

02 Audit чува одлуке које утичу на одговорност, док лог чува технички контекст.

03 Конфигурација одређује очекивано понашање, као што су рокови и прагови, и мора бити проверљива.

ДРУГИ ДЕО

02

Дистрибуирани ток

Шема тока и гаранције које повезују компоненте система.

02 · ТОК

Шема следљивог дистрибуираног тока

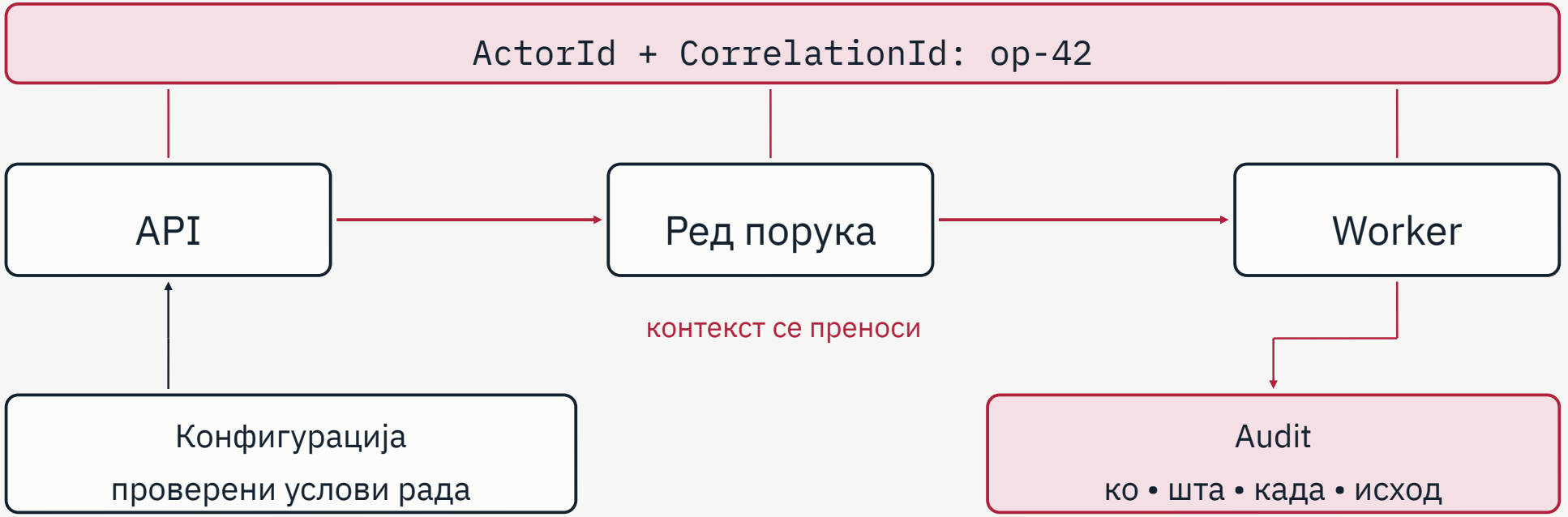
Један контекст пролази кроз све компоненте, па се неуспех може објаснити без нагађања.

Елемент	Улога у току
Улазни захтев	идентитет и корелација
API	прихвата или одбија
Порука и worker	обрађују исти контекст
Audit и траг	објашњавају исход

02 · ДИЈАГРАМ

Један контекст повезује цео ток

Идентитет и CorrelationId се преносе, а Audit и конфигурација објашњавају одлуку.



ТРЕЋИ ДЕО

03

Код и failure тестови

Покретање примера, доменско правило, тест и видљиви исходи.

Покретање примера

Пример прати једну операцију кроз две компоненте.

terminal · Корен Git репозиторијума nastava.

```
cd "03 Osnove Distribuiranog Programiranja/04 Primeri/03 Identitet audit konfiguracija"
dotnet run --project src/Odp.Vezba03.ConsoleUi
dotnet test Odp.Vezba03.sln
```

03 · КОД И ТЕСТ

Правило у коду

Исти CorrelationId види се у логу станице и у ревизијском трагу центра.

C# · доменско правило

```
public ScheduleContactResult Schedule(ContactRequest request, OperationContext context)
{
    ArgumentNullException.ThrowIfNull(request);
    ArgumentNullException.ThrowIfNull(context);
    var code = Decide(request, context);
    auditLog.Record(new AuditEntry(
        clock.UtcNow,
        context.CorrelationId,
        context.ActorId,
        Action,
        request.StationId,
        code));
    return new ScheduleContactResult(code == ContactCodes.Scheduled, code, context.CorrelationId);
}
```

03 · КОД И ТЕСТ

Контекст сценарија

Тест који задаје квар

Ревизијски запис настаје и када је захтев одбијен пре позива станице.

ПРИКАЗАНИ ПРИМЕР

Тест показује да успешан захтев преноси исти CorrelationId до станице и ревизијског трага.

03 · КОД И ТЕСТ

Тест који задаје квар

Тест показује да успешан захтев преноси исти CorrelationId до станице и ревизијског трага.

C# · NUnit

```
[Test]
public void Schedule_CarriesTheSameCorrelationIdToTheStationAndTheAuditTrail()
{
    var result = _handler.Schedule(Request, Context);
    using (Assert.EnterMultipleScope())
    {
        Assert.That(result, Is.EqualTo(new ScheduleContactResult(true, ContactCodes.Scheduled,
            "op-42")));
        Assert.That(_stationGateway.Calls.Single().CorrelationId, Is.EqualTo("op-42"));
        Assert.That(
            _auditLog.Entries.Single(),
            Is.EqualTo(new AuditEntry(
                Now,
                "op-42",
```

Тест који задаје квар (наставак)

Тест показује да успешан захтев преноси исти CorrelationId до станице и ревизијског трага.

C# · NUnit

```
15                  "ana",
16                  "contact.schedule",
17                  "GS-NOVI-SAD",
18                  ContactCodes.Scheduled)));
19      }
20  }
```

Услов failure теста

Audit постоји и за одбијен захтев

Лимит је 15 минута, а захтев траје 40 минута. SetUp припрема исти handler, audit log и station gateway као у претходном тесту.

Audit постоји и за одбијен захтев

Невалидан захтев добија duration_too_long, станица се не позива, а одбијање остаје у ревизијском трагу.

C# · NUnit

```
1   [Test]
2   public void Schedule_WhenRequestIsInvalid_AuditsTheRejectionWithoutCallingTheStation()
3   {
4       var result = _handler.Schedule(Request with { Duration = TimeSpan.FromMinutes(40) }, Context);
5       using (Assert.EnterMultipleScope())
6       {
7           Assert.That(result.Scheduled, Is.False);
8           Assert.That(_stationGateway.Calls, Is.Empty);
9           Assert.That(_auditLog.Entries.Single().Outcome, Is.EqualTo(ContactCodes.DurationTooLong));
10      }
11  }
```

Контекст сценарија

Испис демонстрације

Неисправна конфигурација зауставља покретање са кодом грешке.

ПРИКАЗАНИ ПРИМЕР

Испис повезује успешан контакт и два неуспешна исхода са одговарајућим корелационим идентификаторима.

Испис демонстрације

Испис повезује успешан контакт и два неуспешна исхода са одговарајућим корелационим идентификаторима.

console · исходи демонстрације

```
op-0001: GS-NOVI-SAD, 10 min -> contact_scheduled
op-0002: GS-NOVI-SAD, 40 min -> duration_too_long
op-0003: GS-BEOGRAD, 10 min -> station_unavailable
Log stanice:
  op-0001 | GS-NOVI-SAD | reserved=True
  op-0003 | GS-BEOGRAD | reserved=False
Revizijski trag:
  op-0001 | ana | contact.schedule | GS-NOVI-SAD | contact_scheduled
  op-0002 | marko | contact.schedule | GS-NOVI-SAD | duration_too_long
  op-0003 | jelena | contact.schedule | GS-BEOGRAD | station_unavailable
```

Невалидна конфигурација зауставља покретање

MaxDuration мора бити већи од нуле и највише 60 минута. Провера се извршава пре конструисања use case-а.

01 MaxDuration = 0 → config_max_duration_must_be_positive

02 MaxDuration = 61 min → config_max_duration_above_hard_limit

03 Покретање → TryCreateDemo враћа Demo = null, а Program исписује код грешке и враћа exit code 1.

Невалидна конфигурација зауставља покретање · тест

MaxDuration мора бити већи од нуле и највише 60 минута. Провера се извршава пре конструисања use case-а.

C# · NUnit

```
1   [Test]
2   public void Validate_WhenLimitIsNotPositive_Rejects()
3   {
4       var result = _validator.Validate(new ContactLimits(TimeSpan.Zero));
5       Assert.That(result.Error, Is.EqualTo(ConfigurationCodes.MaxDurationMustBePositive));
6   }
```

ЧЕТВРТИ ДЕО

04

Примена и провера

Принципи, замке, задатак уз пример и примена на пројектну целину.

04 · ПРИМЕНА

Принципи и замке

Принцип важи тек када је препозната и замка која га тихо поништава.

ПРИНЦИПИ

- Audit чува одлуке које су важне за одговорност и истрагу.
- Логови и трагови чувају технички контекст, али не тајне.
- Конфигурација има валидно значење, власника и начин промене.

ЗАМКЕ

- Лог који открива креденцијале или личне податке.
- Корелациони идентификатор креиран у свакој компоненти уместо пренет.
- Конфигурација прихваћена без провере опсега и утицаја.

04 · ПРИМЕНА

Типичне грешке

Ове грешке се најчешће понављају када се принцип површно примени.

01 Тајне у логовима.

02 Нови корелациони идентификатор у сваком слоју.

03 Конфигурација без валидног опсега.

04 · ПРИМЕНА

Како се принцип примењује

Кораци показују како се принцип препознаје и доследно спроводи у развоју.

01 Пратити неколико важних токова од захтева до исхода.

02 Преносити исти контекст кроз границе.

03 Тестирати промењен или невалидан услов рада.

04 · ПРИМЕНА

Питање за дискусију

Одговор се образлаже failure сценаријем и тестом, а не описом срећног пута.

Како бисте за један неуспешан захтев спојили траг из API-ја, реда порука и workera?

ЗА ВЕЖБУ

Како бисте реконструисали једну неуспелу операцију која је прошла кроз API, ред порука и worker, а да не читате неограничену количину логова?

04 · ПРИМЕНА

Задатак уз пример

Проширење примера уз вежбу, правило иде у домен, а доказ у тест.

01 Додати у OperationContext идентификатор позивајуће компоненте (Source) и уписати га у ревизијски запис.

02 Проширити тест тако да доказује да запис садржи и извор операције.

04 · ПРИМЕНА

Примена на пројекат

Принцип са вежбе примењује се на додељену пројектну целину.

01 Идентификовати конкретну дистрибуирану одлуку у својој целини.

02 Имплементирати је са стабилним кодом исхода.

03 Припремити поновљив failure сценарио и тест који га доказује.

04 · ПРИМЕНА

Резиме

Тим бира неколико операција које мора умети да прати од захтева до исхода.

Један контекст пролази кроз све компоненте, па се неуспех може објаснити без нагађања.

НАПОМЕНА

Наредна вежба: Верзионисање и failure-first тестирање.